call code. exit code, entry code for exam explain like in ppt

explain in an axiomatic way: Definition + interrelationship.

Don't be ambiguous!

Understand what is asked!

Conclusion

build the new stackframe

theory

schematic source code example:

```
push ebp
mov ebp, esp
```

entry code { .... call code .... } exit code

You have to explain what every step is doing

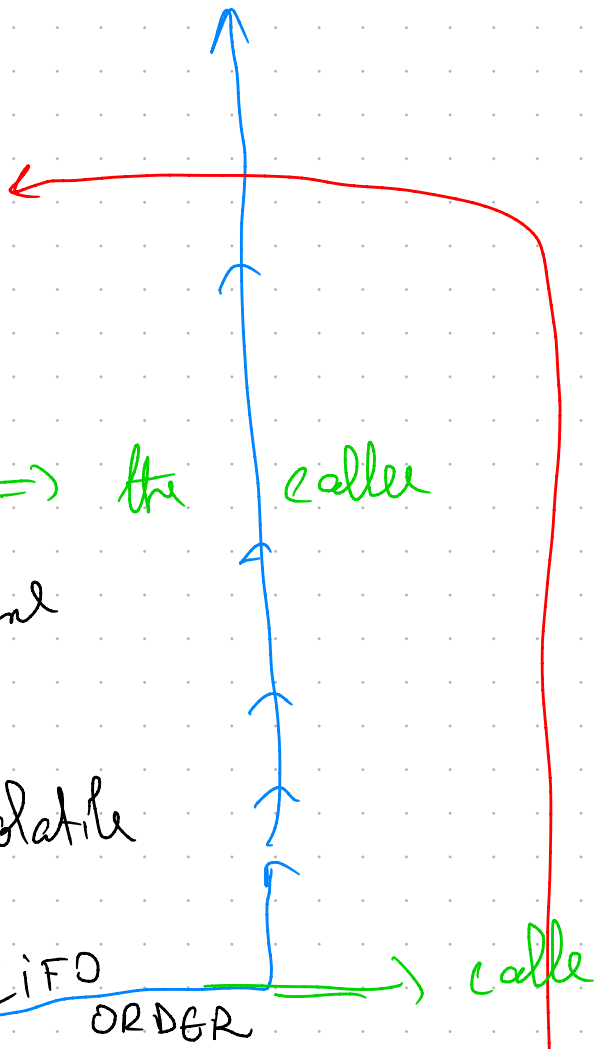
: stack LIFO PACK; UNPACK, etc....

Call code ⟹ generated by the caller

a) - volatile

b) - params

c) - call



Entry code ⟹ the callee

a) new stackframe

b) local vars.

c) save non-volatile

Exit code LIFO ORDER ⟹ calle

"It reverses all the actions"

a) restore non volatile

b) free local variables

c) dealloc. stack frame

d) returning from the subroutine

ret + [maybe freeing params] iff STDCALL. the

The caller is resp. for restoring volatile vars.

(not part of any call code or exit code) it's just common sense.

This is how u need to explain at the exam, not just enumerations.

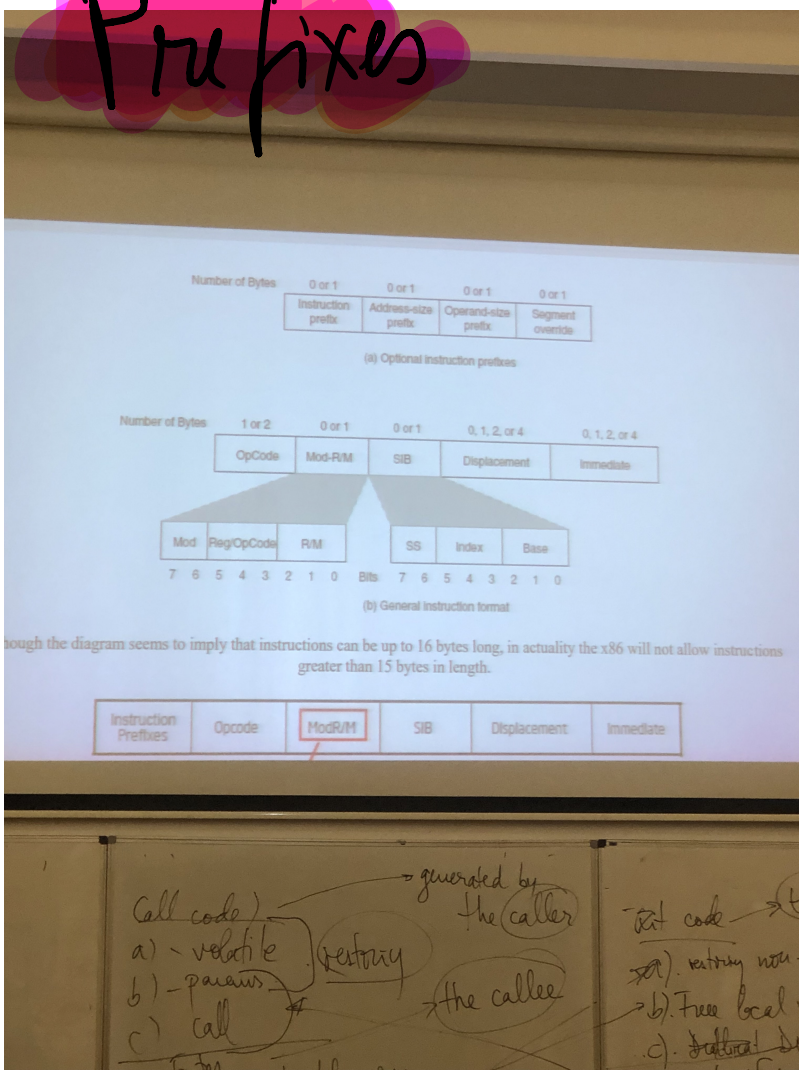
Caller	Callee	Func/proc. call	{	}
		Call code	Entry code	Exit code
C	C	C compiler	C compiler	C compiler
C	asm	C compiler	ASM programmer	ASM programmer
asm	C	ASM programmer	C compiler	C compiler
asm	asm	call instruction	NOTHING MANDATORY	RET

↳ - You don't have volatile vars.

- You don't need a new stack frame
- no local vars
-

Explain the convention CDECL, STDCALL

Prefixes



The first 4 bytes are prefixes.

a) Instruction prefixes → explicit prefixes
   REP movsb
   given by the programmer

b) Segment prefixes
   ES xlat
   ← if you want to change the implicit ESX (segment register) for that particular instruction

c) operand - size prefix
d) Address - size prefix
} implicit, generated by the assembler

F3h – REP, REPE – (repeat while equal)
F2h – REPNE – (repeat while ! equal)

REP instruction (make your own)
↓
F3h [instruction-code] for the exam.

2E h = CS
36 h = SS
3E h = DS
26 h = ES

ES xlat 26:D7 (ES:EBX)
mov eax, [ebx] 8B03
mov eax, [SS:ebx] - 36:8B03
mov eax, [CS:ebx] - 2E:8B03

ES lodsb 26:AC ; LODS byte ptr ES:[ESI]
movsb A4 ; DS:[ESI]
ES movsb 26:A4 ; ES:[ESI]

66h – operand override
67h – address override

bits 32
cbw ; 66:98 ← prefix; it just says that result is not 32 bits
cwd ; 66:99
cwde ; 98

bits 16
cbw ; 98
cwde ; 66:98

```
push ax  ; 66:50
push eax ; 50
```

you are not consistent w/ the "bits 32" directive

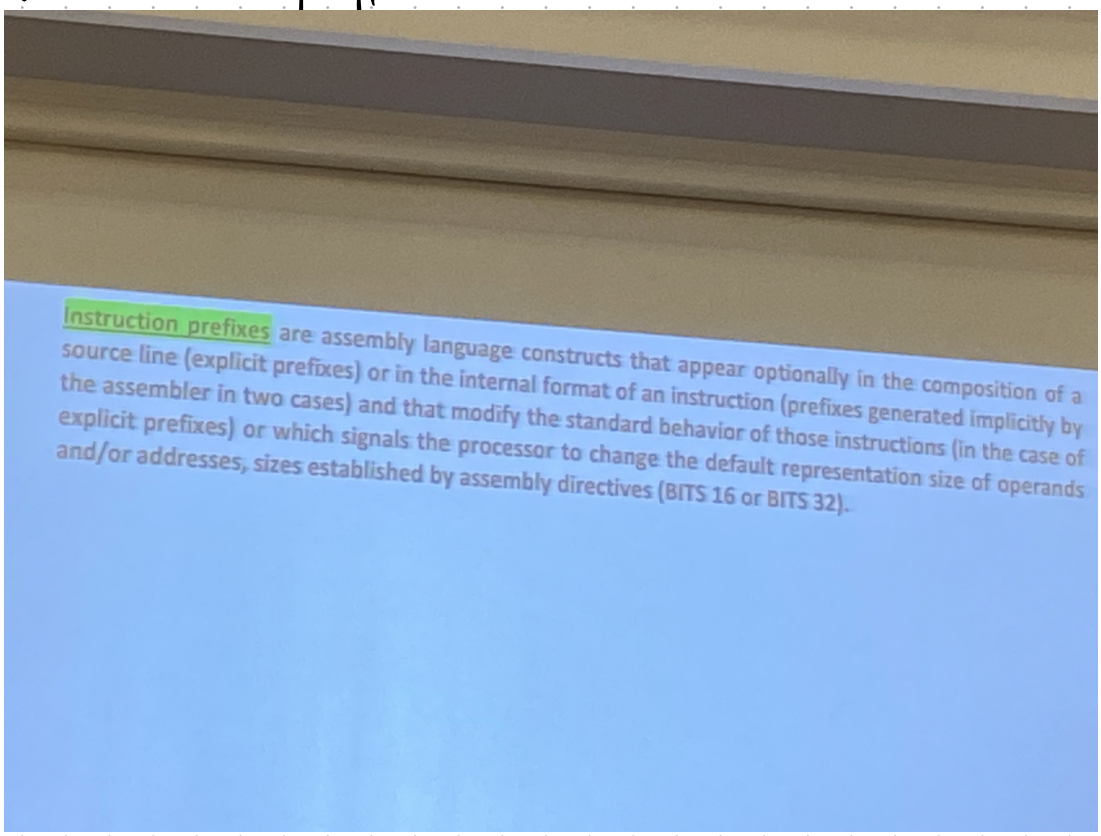
Operand size prefix - 0x66

```
bits 16
push dword [ebx] ; 66:67:FF33
```

- addressing of [ebx] is on 32 bits => 67
- dword instead of word => 66

```
rep push dword [cs:ebx] ; F3:2E:66:67:FF33
```

Instruction prefixes



Conversion classification

- Destructive:
  - cbw
  - cwd
  - cwde
  - cdq
  - movzx
  - movsx
  - mov ah,0 ...

  proper examples

- non-destructive:
  - byte
  - word
  - dword
  - qword

- signed: (they take the s bit into consideration)
  - cbw
  - cwd
  - cwde

- unsigned (opposite)
  - movzx
  - mov dh,0 ...

- by enlargement
  - all destructive ops + word, dword, qword
- by narrowing
  byte, word, dword.

I Theory (concise that you know the definition)
- from the courses
- microprocessor 8086
- 8 general registers
- segment -"-
- address computation
- 81000h?
- offset specific formula
- near address, far address
- basics of asm lang
- location counter
- instructions, keywords, label, location counter.
- ETC. with comments / very detailed / explained + linked w/ theory
- all give examples illustrating the theory.

- 2's complement
- when do we need it? when does it make sense?
- working w/ negative numbers.
- overflow concept (mathematical +, ~)
  admissible representation interval.

  Call n operations +, ~, *, /
  OF, CF related to the concept.

- why do we have imul and idiv, but not iadd / isub.
- little endian
- what, else

IV: problem: normal / multi-modal
- conversions
- working w bits
- arithmetic operations
- string operations
- printf, scanf

II data segment base 16.

III code segment
- 3 same codes / explain what they are doing / what they change

v reno 2

```
add ebx, v
sub ebx, 6
mov eax, ebx
```

with one single instruct. to have the same effect:

-11- to have the same effect to eax:

" mov eax, ebx + v - 6 " + explain

```
lea eax, [ebx + v - 6]
```

v reno 2

```
add ebx, v
sub ebx, 6
mov eax, [ebx]
```

⇓⇓

```
mov eax, [ebx + v - 6]
```

(probably a memory allocation error, best)

v reverse 2

```
add ebx [v]
sub ebx, 6
mov ecx, ebx
```

NO SOLUTION: (no equivalent in offset specification formula - which is the only way to express an instruction)